MODULE 10

GitHub Repository Link: https://github.com/devcnx/csd440

Source Code

BrittaneyJSON.php

```
<?php
declare(strict_types=1);

/**
 * Module 10.2 Programming Assignment — JSON Form
 * CSD440 Server-Side Scripting
 *
 * Presents a form that collects at least 8 fields of user data. The form
 * submits to BrittaneyJSONResponse.php, which validates, encodes the data
 * into JSON using json_encode(), and displays the result. If validation
 * fails, errors are stored in the session and the user is redirected back
 * to this form with error messages and repopulated fields.
 *
 * Form fields:
 *   - full_name:            required, max 100 chars
 *   - email:                required, valid email
 *   - phone:                required, 10 digits, formatted (XXX) XXX-XXXX
 *   - age:                  required, integer 1-120
 *   - city:                 required, max 100 chars
 *   - state:                required, US state abbreviation (whitelist)
 *   - programming_language: required, whitelist of languages
 *   - experience_years:     required, integer 0-80
 *   - bio:                  required, max 500 chars
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-16
 * @php     >= 8.0
 */

session_start([
    'cookie_httponly' => true,
    'cookie_samesite' => 'Strict',
]);
require_once __DIR__ . '/functions.php';

// Retrieve flash data from session (set by BrittaneyJSONResponse on validation
failure)
$errors  = $_SESSION['form_errors'] ?? [];
$oldData = $_SESSION['form_data'] ?? [];
unset($_SESSION['form_errors'], $_SESSION['form_data']);
```

```
// Generate CSRF token for form protection
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// Page metadata
date_default_timezone_set('America/Chicago');
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 10.2 Programming Assignment — JSON Form';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JSON Form — <?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-
8'); ?></title>
    <link rel="stylesheet" href="../../shared.css">
</head>

<body>
    <h1>JSON Data Form</h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8');
?></strong></p>
        <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> —
<?php echo htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
        <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

<?php if (!empty($errors)): ?>
    <div class="error-summary">
        <h2>Correction Required</h2>
        <p class="error-message">Please fix the following issues and resubmit:</p>
        <ul class="error-list">
<?php foreach ($errors as $message): ?>
            <li><?php echo htmlspecialchars($message, ENT_QUOTES, 'UTF-8'); ?></li>
<?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

    <form method="POST" action="BrittaneyJSONResponse.php" class="registration-form">
        <input type="hidden" name="csrf_token" value="<?php echo
htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8'); ?>">
        <div class="form-group">
            <label for="full_name">Full Name <span class="required">*</span></label>
            <input type="text" id="full_name" name="full_name"
                   placeholder="e.g., Jane Doe" maxlength="100"
                   value="<?php echo old('full_name', $oldData); ?>" required>
<?php if (isset($errors['full_name'])): ?>
```

```
            <span class="error"><?php echo htmlspecialchars($errors['full_name'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="email">Email <span class="required">*</span></label>
            <input type="email" id="email" name="email"
                   placeholder="e.g., jane@example.com"
                   value="<?php echo old('email', $oldData); ?>" required>
<?php if (isset($errors['email'])): ?>
            <span class="error"><?php echo htmlspecialchars($errors['email'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="phone">Phone <span class="required">*</span></label>
            <input type="tel" id="phone" name="phone"
                   placeholder="e.g., (555) 123-4567" maxlength="14"
                   value="<?php echo old('phone', $oldData); ?>" required>
<?php if (isset($errors['phone'])): ?>
            <span class="error"><?php echo htmlspecialchars($errors['phone'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="age">Age <span class="required">*</span></label>
            <input type="number" id="age" name="age"
                   min="1" max="120" placeholder="e.g., 30"
                   value="<?php echo old('age', $oldData); ?>" required>
<?php if (isset($errors['age'])): ?>
            <span class="error"><?php echo htmlspecialchars($errors['age'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="city">City <span class="required">*</span></label>
            <input type="text" id="city" name="city"
                   placeholder="e.g., Omaha" maxlength="100"
                   value="<?php echo old('city', $oldData); ?>" required>
<?php if (isset($errors['city'])): ?>
            <span class="error"><?php echo htmlspecialchars($errors['city'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="state">State <span class="required">*</span></label>
            <select id="state" name="state" required>
                <option value="">— Select —</option>
<?php foreach (US_STATES as $st): ?>
                <option value="<?php echo $st; ?>"<?php echo selected('state', $st,
```

```
$oldData); ?>><?php echo $st; ?></option>
<?php endforeach; ?>
            </select>
<?php if (isset($errors['state'])): ?>
            <span class="error"><?php echo htmlspecialchars($errors['state'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="programming_language">Programming Language <span
class="required">*</span></label>
            <select id="programming_language" name="programming_language" required>
                <option value="">— Select —</option>
<?php foreach (PROGRAMMING_LANGUAGES as $lang): ?>
                <option value="<?php echo $lang; ?>"<?php echo
selected('programming_language', $lang, $oldData); ?>><?php echo $lang; ?></option>
<?php endforeach; ?>
            </select>
<?php if (isset($errors['programming_language'])): ?>
            <span class="error"><?php echo
htmlspecialchars($errors['programming_language'], ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="experience_years">Years of Experience <span
class="required">*</span></label>
            <input type="number" id="experience_years" name="experience_years"
                   min="0" max="80" placeholder="e.g., 5"
                   value="<?php echo old('experience_years', $oldData); ?>" required>
<?php if (isset($errors['experience_years'])): ?>
            <span class="error"><?php echo
htmlspecialchars($errors['experience_years'], ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-group">
            <label for="bio">Bio <span class="required">*</span></label>
            <textarea id="bio" name="bio" rows="4" maxlength="500"
                      placeholder="Tell us about yourself..." required><?php echo
old('bio', $oldData); ?></textarea>
<?php if (isset($errors['bio'])): ?>
            <span class="error"><?php echo htmlspecialchars($errors['bio'],
ENT_QUOTES, 'UTF-8'); ?></span>
<?php endif; ?>
        </div>

        <div class="form-actions">
            <button type="submit">Encode as JSON</button>
            <a href="BrittaneyJSON.php" class="btn-secondary">Reset</a>
        </div>

        <p class="form-note"><span class="required">*</span> Required fields</p>
    </form>
```

```
</body>

</html>
```

BrittaneyJSONResponse.php

```
<?php
declare(strict_types=1);

/**
 * Module 10.2 Programming Assignment — JSON Response Handler
 * CSD440 Server-Side Scripting
 *
 * Receives form data from BrittaneyJSON.php, validates all fields server-side,
 * encodes the data into JSON format using json_encode(), and displays the
 * result in a well-formatted output. If validation fails, stores errors and
 * submitted data in the session and redirects back to the form.
 *
 * Form fields validated:
 *   - full_name:            required, max 100 chars
 *   - email:                required, valid email (FILTER_VALIDATE_EMAIL)
 *   - phone:                required, 10 digits, formatted (XXX) XXX-XXXX
 *   - age:                  required, integer 1–120
 *   - city:                 required, max 100 chars
 *   - state:                required, US state abbreviation (whitelist)
 *   - programming_language: required, whitelist of languages
 *   - experience_years:     required, integer 0–80
 *   - bio:                  required, max 500 chars
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-16
 * @php     >= 8.0
 */

session_start([
    'cookie_httponly' => true,
    'cookie_samesite' => 'Strict',
]);
require_once __DIR__ . '/functions.php';

date_default_timezone_set('America/Chicago');

/**
 * Renders the JSON confirmation page after successful validation.
 *
 * Displays all submitted data in a formatted table and the raw JSON
 * output produced by json_encode() with JSON_PRETTY_PRINT.
 *
 * @param  array $sanitizedData HTML-escaped data for table display.
 * @param  array $jsonData      Raw data for JSON encoding.
 * @return void
 */
```

```
function renderConfirmation(array $sanitizedData, array $jsonData): void
{
    try {
        $jsonEncoded = json_encode($jsonData, JSON_PRETTY_PRINT |
JSON_THROW_ON_ERROR);
    } catch (JsonException $e) {
        $jsonEncoded = 'Error encoding data: ' . htmlspecialchars($e->getMessage(),
ENT_QUOTES, 'UTF-8');
    }
?>
    <div class="confirmation">
        <h2>Data Encoded Successfully</h2>
        <p class="success-message">Your form data has been encoded into JSON format
using <code>json_encode()</code>.</p>

        <table class="data-display">
            <caption>Submitted Data</caption>
            <tbody>
                <tr>
                    <td>Full Name</td>
                    <td><?php echo $sanitizedData['full_name']; ?></td>
                </tr>
                <tr>
                    <td>Email</td>
                    <td><code><?php echo $sanitizedData['email']; ?></code></td>
                </tr>
                <tr>
                    <td>Phone</td>
                    <td><?php echo $sanitizedData['phone']; ?></td>
                </tr>
                <tr>
                    <td>Age</td>
                    <td><?php echo $sanitizedData['age']; ?></td>
                </tr>
                <tr>
                    <td>City</td>
                    <td><?php echo $sanitizedData['city']; ?></td>
                </tr>
                <tr>
                    <td>State</td>
                    <td><?php echo $sanitizedData['state']; ?></td>
                </tr>
                <tr>
                    <td>Programming Language</td>
                    <td><?php echo $sanitizedData['programming_language']; ?></td>
                </tr>
                <tr>
                    <td>Experience (Years)</td>
                    <td><?php echo $sanitizedData['experience_years']; ?></td>
                </tr>
                <tr>
                    <td>Bio</td>
                    <td><?php echo nl2br($sanitizedData['bio']); ?></td>
                </tr>
            </tbody>
```

```
        </table>
    </div>

    <div class="confirmation json-output">
        <h2>JSON Output</h2>
        <p class="success-message">Encoded with <code>json_encode($data,
JSON_PRETTY_PRINT)</code></p>
        <pre><code><?php echo htmlspecialchars($jsonEncoded, ENT_QUOTES, 'UTF-8');
?></code></pre>
    </div>

    <div class="form-actions">
        <a href="BrittaneyJSON.php">Submit Another Entry</a>
    </div>
<?php
}

// Page metadata
$studentName     = 'Brittaney Perry-Morgan';
$assignmentTitle = 'Module 10.2 Programming Assignment — JSON Response';
$courseName      = 'CSD440 Server-Side Scripting';
$today           = date('F j, Y');

// Initialize variables
$errors  = [];
$isValid = false;

// Verify form was submitted via POST
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    // Redirect to form if accessed directly
    header('Location: BrittaneyJSON.php');
    exit;
}

// Verify CSRF token to prevent cross-site request forgery
if (!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'] ?? '',
$_POST['csrf_token'])) {
    $_SESSION['form_errors'] = ['csrf' => 'Invalid form submission. Please try
again.'];
    $_SESSION['form_data'] = [];
    header('Location: BrittaneyJSON.php');
    exit;
}

// Collect and validate form data
$errors  = validateFormData($_POST);
$isValid = empty($errors);
?>
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?php echo $isValid ? 'JSON Output' : 'Correction Required'; ?> — <?php
```

```
echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?></title>
    <link rel="stylesheet" href="../../shared.css">
</head>

<body>
    <h1><?php echo $isValid ? 'JSON Output' : 'Form Submission Error'; ?></h1>

    <div class="header-info">
        <p><strong><?php echo htmlspecialchars($studentName, ENT_QUOTES, 'UTF-8');
?></strong></p>
        <p><?php echo htmlspecialchars($assignmentTitle, ENT_QUOTES, 'UTF-8'); ?> —
<?php echo htmlspecialchars($courseName, ENT_QUOTES, 'UTF-8'); ?></p>
        <p><?php echo htmlspecialchars($today, ENT_QUOTES, 'UTF-8'); ?></p>
    </div>

<?php
if ($isValid):
    // Prepare sanitized data for HTML display
    $sanitizedData = [
        'full_name'            => validateText($_POST['full_name']),
        'email'                => validateEmail($_POST['email']),
        'phone'                => validatePhone($_POST['phone']),
        'age'                  => validateInteger($_POST['age'], 1, 120),
        'city'                 => validateText($_POST['city']),
        'state'                => validateSelect($_POST['state'], US_STATES),
        'programming_language' => validateSelect($_POST['programming_language'],
PROGRAMMING_LANGUAGES),
        'experience_years'     => validateInteger($_POST['experience_years'], 0, 80),
        'bio'                  => validateTextarea($_POST['bio'], 500),
    ];

    // Build raw data for JSON encoding (no HTML escaping, proper types)
    $jsonData = buildJsonData($_POST);

    renderConfirmation($sanitizedData, $jsonData);
else:
    // Validation failed — store errors and data in session, redirect to form
    $_SESSION['form_errors'] = $errors;
    $_SESSION['form_data'] = $_POST;
    header('Location: BrittaneyJSON.php');
    exit;
endif;
?>

</body>

</html>
```

functions.php

```
<?php
declare(strict_types=1);
```

```php
/**
 * Module 10.2 — Shared Functions and Constants
 * CSD440 Server-Side Scripting
 *
 * Contains validation functions, data formatting helpers, and whitelist
 * constants used by both BrittaneyJSON.php and BrittaneyJSONResponse.php.
 * Included via require_once to avoid duplication.
 *
 * @author  Brittaney Perry-Morgan
 * @date    2026-05-16
 * @php     >= 8.0
 */

// ── Whitelist data for dropdowns ──────────────────────────────
const US_STATES = [
    'AL','AK','AZ','AR','CA','CO','CT','DE','FL','GA',
    'HI','ID','IL','IN','IA','KS','KY','LA','ME','MD',
    'MA','MI','MN','MS','MO','MT','NE','NV','NH','NJ',
    'NM','NY','NC','ND','OH','OK','OR','PA','RI','SC',
    'SD','TN','TX','UT','VT','VA','WA','WV','WI','WY',
];

const PROGRAMMING_LANGUAGES = [
    'C', 'C++', 'C#', 'Go', 'Java', 'JavaScript',
    'Kotlin', 'PHP', 'Python', 'Ruby', 'Rust', 'Swift', 'TypeScript',
];

/**
 * Validates and sanitizes text input.
 *
 * Trims whitespace and applies htmlspecialchars() for safe display.
 * Returns null if the input is empty after trimming.
 *
 * @param  string       $input The raw input value.
 * @return string|null         Sanitized string, or null if empty.
 */
function validateText(string $input): ?string
{
    $trimmed = trim($input);
    return $trimmed === '' ? null : htmlspecialchars($trimmed, ENT_QUOTES, 'UTF-8');
}

/**
 * Validates an email address.
 *
 * Uses PHP's FILTER_VALIDATE_EMAIL to check format. Returns the sanitized
 * email if valid, null otherwise.
 *
 * @param  string       $email The raw email input.
 * @return string|null         Sanitized email, or null if invalid.
 */
function validateEmail(string $email): ?string
{
    $trimmed = trim($email);
    if ($trimmed === '' || !filter_var($trimmed, FILTER_VALIDATE_EMAIL)) {
```

```
        return null;
    }
    return htmlspecialchars($trimmed, ENT_QUOTES, 'UTF-8');
}

/**
 * Validates an integer within a specified range.
 *
 * Checks that the input is a valid integer and falls within the min/max
 * bounds. Returns the integer if valid, null otherwise.
 *
 * @param  string    $input The raw input value.
 * @param  int       $min   Minimum acceptable value (inclusive).
 * @param  int       $max   Maximum acceptable value (inclusive).
 * @return int|null          Validated integer, or null if invalid.
 */
function validateInteger(string $input, int $min, int $max): ?int
{
    $trimmed = trim($input);
    if ($trimmed === '') {
        return null;
    }
    $int = filter_var($trimmed, FILTER_VALIDATE_INT);
    if ($int === false || $int < $min || $int > $max) {
        return null;
    }
    return $int;
}

/**
 * Validates a phone number format.
 *
 * Accepts common US phone formats. Returns a standardized (XXX) XXX-XXXX
 * format if exactly 10 digits are found, null otherwise.
 *
 * @param  string       $phone The raw phone input.
 * @return string|null         Formatted phone number, or null if invalid.
 */
function validatePhone(string $phone): ?string
{
    $trimmed = trim($phone);
    $digits = preg_replace('/\D/', '', $trimmed);
    if (strlen($digits) !== 10) {
        return null;
    }
    return sprintf('(%s) %s-%s',
        substr($digits, 0, 3),
        substr($digits, 3, 3),
        substr($digits, 6, 4)
    );
}

/**
 * Validates a select dropdown value against allowed options.
 *
```

```
 * Checks that the submitted value exists in the whitelist of acceptable
 * options. Returns the sanitized value if valid, null otherwise.
 *
 * @param  string       $input   The raw select value.
 * @param  array       $options Whitelist of acceptable values.
 * @return string|null           Validated option, or null if invalid.
 */
function validateSelect(string $input, array $options): ?string
{
    $trimmed = trim($input);
    if ($trimmed === '' || !in_array($trimmed, $options, true)) {
        return null;
    }
    return htmlspecialchars($trimmed, ENT_QUOTES, 'UTF-8');
}

/**
 * Validates a textarea input with length limits.
 *
 * Trims whitespace and enforces a maximum character count. Returns the
 * sanitized text if valid, null if empty or too long.
 *
 * @param  string       $input  The raw textarea input.
 * @param  int          $maxLen Maximum allowed characters.
 * @return string|null          Sanitized text, or null if invalid.
 */
function validateTextarea(string $input, int $maxLen): ?string
{
    $trimmed = trim($input);
    if ($trimmed === '') {
        return null;
    }
    if (strlen($trimmed) > $maxLen) {
        return null;
    }
    return htmlspecialchars($trimmed, ENT_QUOTES, 'UTF-8');
}

/**
 * Collects validation errors for all form fields.
 *
 * Returns an associative array where keys are field names and values
 * are error messages. An empty array indicates all fields passed validation.
 *
 * @param  array $data The raw POST data.
 * @return array       Array of error messages (empty if all valid).
 */
function validateFormData(array $data): array
{
    $errors = [];

    // Full name — required text, max 100 chars
    if (validateText($data['full_name'] ?? '') === null) {
        $errors['full_name'] = 'Full name is required.';
    } elseif (strlen(trim($data['full_name'])) > 100) {
```

```
        $errors['full_name'] = 'Full name must be 100 characters or fewer.';
    }

    // Email — required, valid format
    if (validateEmail($data['email'] ?? '') === null) {
        $errors['email'] = 'A valid email address is required.';
    }

    // Phone — required, 10 digits
    if (validatePhone($data['phone'] ?? '') === null) {
        $errors['phone'] = 'A valid 10-digit phone number is required.';
    }

    // Age — required integer, 1-120
    if (validateInteger($data['age'] ?? '', 1, 120) === null) {
        $errors['age'] = 'Age is required and must be between 1 and 120.';
    }

    // City — required text, max 100 chars
    if (validateText($data['city'] ?? '') === null) {
        $errors['city'] = 'City is required.';
    } elseif (strlen(trim($data['city'])) > 100) {
        $errors['city'] = 'City must be 100 characters or fewer.';
    }

    // State — required select from whitelist
    if (validateSelect($data['state'] ?? '', US_STATES) === null) {
        $errors['state'] = 'Please select a valid state.';
    }

    // Programming language — required select from whitelist
    if (validateSelect($data['programming_language'] ?? '', PROGRAMMING_LANGUAGES) ===
null) {
        $errors['programming_language'] = 'Please select a valid programming
language.';
    }

    // Experience years — required integer, 0-80
    if (validateInteger($data['experience_years'] ?? '', 0, 80) === null) {
        $errors['experience_years'] = 'Years of experience is required (0-80).';
    }

    // Bio — required textarea, max 500 chars
    if (validateTextarea($data['bio'] ?? '', 500) === null) {
        $errors['bio'] = 'Bio is required (max 500 characters).';
    }

    return $errors;
}

/**
 * Builds a data array with proper types for JSON encoding.
 *
 * Numeric fields are cast to int so JSON output shows them as numbers,
 * not strings. Phone is formatted for display. All other fields are
```

```
 * trimmed raw strings (not HTML-escaped) to produce valid JSON.
 *
 * @param  array $postData The raw POST data.
 * @return array Data ready for json_encode().
 */
function buildJsonData(array $postData): array
{
    $phone = trim($postData['phone'] ?? '');
    $digits = preg_replace('/\D/', '', $phone);
    $formattedPhone = strlen($digits) === 10
        ? sprintf('(%s) %s-%s', substr($digits, 0, 3), substr($digits, 3, 3),
substr($digits, 6, 4))
        : $phone;

    return [
        'full_name'            => trim($postData['full_name'] ?? ''),
        'email'                => trim($postData['email'] ?? ''),
        'phone'                => $formattedPhone,
        'age'                  => (int)trim($postData['age'] ?? '0'),
        'city'                 => trim($postData['city'] ?? ''),
        'state'                => trim($postData['state'] ?? ''),
        'programming_language' => trim($postData['programming_language'] ?? ''),
        'experience_years'     => (int)trim($postData['experience_years'] ?? '0'),
        'bio'                  => trim($postData['bio'] ?? ''),
    ];
}

/**
 * Returns old form data for a field, HTML-escaped.
 *
 * @param  string $field    The field name.
 * @param  array  $oldData  The stored old data.
 * @return string           Escaped value or empty string.
 */
function old(string $field, array $oldData): string
{
    return isset($oldData[$field]) ? htmlspecialchars(trim($oldData[$field]),
ENT_QUOTES, 'UTF-8') : '';
}

/**
 * Returns 'selected' if a dropdown option matches the old value.
 *
 * @param  string $field    The field name.
 * @param  string $value    The option value to check.
 * @param  array  $oldData  The stored old data.
 * @return string           'selected' or empty string.
 */
function selected(string $field, string $value, array $oldData): string
{
    return (isset($oldData[$field]) && trim($oldData[$field]) === $value) ? '
selected' : '';
}
```